Multi-User AI Chatbot with Login & Persistent Conversation History

1. Main Goal

In this application, we combine several concepts:

User Signup

+

User Login

+

Multiple Users

+

Persistent User Data

+

Persistent Chat History

+

RunnableWithMessageHistory

+

Session ID

+

LCEL Chain

The final result is:

MULTI-USER AI CHATBOT

Each registered user can:

Signup

↓

Login

↓

Chat with AI

↓

Maintain Separate Chat History

↓

Logout

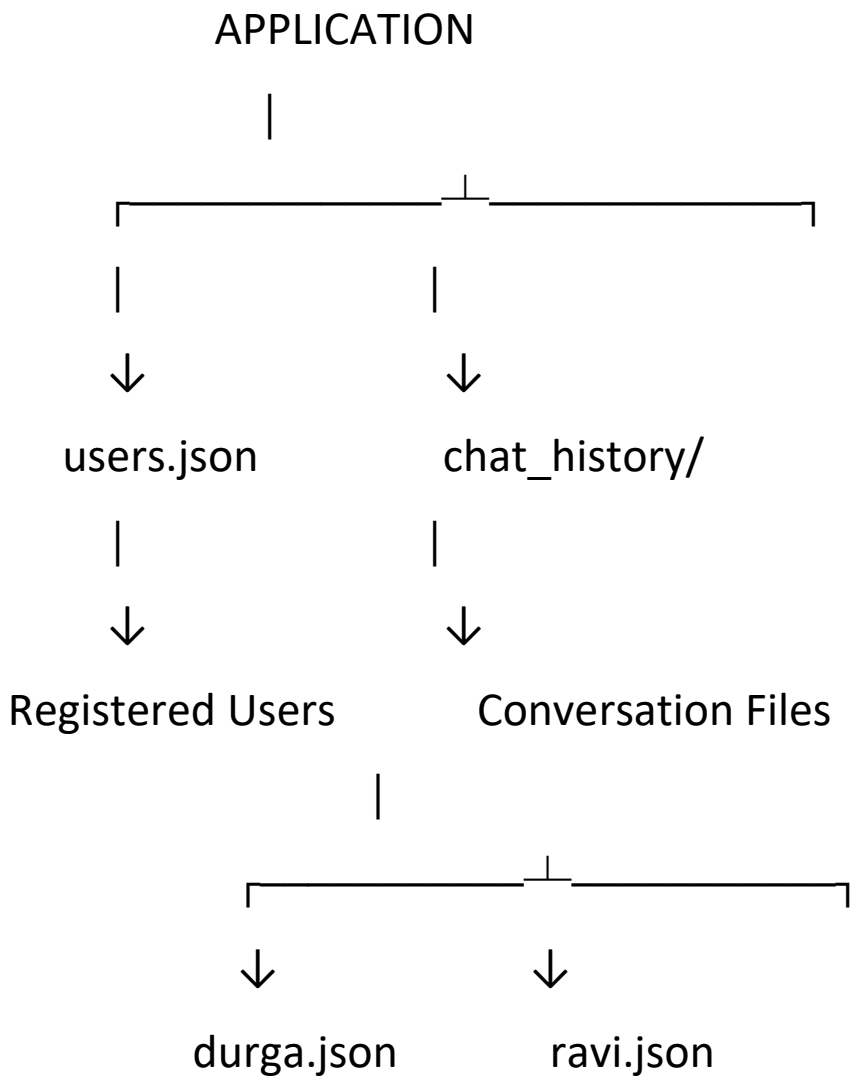
↓

Login Again

↓

Continue Previous Conversation

2. Main Architecture



So we maintain two types of persistent data:

1. User Information

↓

users.json

2. Chat History

↓

chat_history/<username>.json

3. Complete Program

```
import os
import json

from langchain_openai import ChatOpenAI

from langchain_core.prompts import (
    ChatPromptTemplate,
    MessagesPlaceholder
)

from langchain_core.chat_history import (
    BaseChatMessageHistory
)

from langchain_core.messages import (
    message_to_dict,
```

```
    messages_from_dict
)

from langchain_core.runnables.history import (
    RunnableWithMessageHistory
)



# ========================================
# 1. STORAGE CONFIGURATION
# ========================================

USERS_FILE = "users.json"

HISTORY_FOLDER = "chat_history"

os.makedirs(
    HISTORY_FOLDER,
    exist_ok=True
)
```

```
# ========================================
# 2. LOAD USERS
# ========================================

def load_users():

    if not os.path.exists(USERS_FILE):
        return {}

    with open(USERS_FILE, "r") as file:
        return json.load(file)



# ========================================
# 3. SAVE USERS
# ========================================

def save_users(users):
```

```
    with open(USERS_FILE, "w") as file:
        json.dump(users, file, indent=2)


# ========================================
# 4. USER SIGNUP
# ========================================

def signup(users):

    print("\n--- SIGNUP ---")

    name = input("Enter Name: ").strip()

    username = input(
        "Enter Username: "
    ).strip().lower()

    password = input("Enter Password: ")
```

```
if not name or not username or not password:
    print("All fields are required")
    return

if not username.replace("_", "").isalnum():
    print(
        "Use only letters, numbers and underscores"
    )
    return

if username in users:
    print("Username already exists")
    return

users[username] = {
    "name": name,
    "password": password
}
```

```
    save_users(users)

    print("Signup Successful!")


# ========================================
# 5. USER LOGIN
# ========================================

def login(users):

    print("\n--- LOGIN ---")

    username = input(
        "Enter Username: "
    ).strip().lower()

    password = input("Enter Password: ")

    if username not in users:
```

```
        print("Invalid Username")
        return None

    if users[username]["password"] == password:

        print("Login Successful!")

        print(
            "Welcome",
            users[username]["name"]
        )

        return username

    print("Incorrect Password")

    return None


# ========================================
```

```
# 6. FILE CHAT MESSAGE HISTORY
# ========================================

class FileChatMessageHistory(
    BaseChatMessageHistory
):

    def __init__(
        self,
        session_id
    ):

        self.file_path = os.path.join(
            HISTORY_FOLDER,
            session_id + ".json"
        )



    # ---------------------------------
    # READ CHAT HISTORY
```

```
# ----------------------------------

@property
def messages(self):

    if not os.path.exists(
        self.file_path
    ):
        return []

    with open(
        self.file_path,
        "r"
    ) as file:

        data = json.load(file)

    return messages_from_dict(
        data
    )
```

```
# ----------------------------------
# SAVE CHAT HISTORY
# ----------------------------------

def add_messages(
    self,
    messages
):

    all_messages = (
        self.messages
        + messages
    )

    data = [
        message_to_dict(message)
        for message in all_messages
    ]
```

```
    with open(
        self.file_path,
        "w"
    ) as file:

        json.dump(
            data,
            file,
            indent=2
        )


# ----------------------------------
# CLEAR CHAT HISTORY
# ----------------------------------

def clear(self):

    with open(
```

```
            self.file_path,
            "w"
        ) as file:

            json.dump(
                [],
                file
            )


# ========================================
# 7. CREATE OPENAI MODEL
# ========================================

llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
# ========================================
# 8. CREATE PROMPT
# ========================================

prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a helpful AI assistant. "
            "Answer in simple English. "
            "The user's name is {name}."
        ),

        MessagesPlaceholder(
            "history"
        ),

        (
            "human",
            "{question}"
```

```
        )
    ]
)


# ========================================
# 9. CREATE CHAIN
# ========================================

chain = (
    prompt
    | llm
)


# ========================================
# 10. GET SESSION HISTORY
# ========================================

def get_session_history(
```

```
    session_id
):

    return FileChatMessageHistory(
        session_id
    )


# ========================================
# 11. CREATE CHATBOT WITH HISTORY
# ========================================

chatbot = RunnableWithMessageHistory(

    chain,

    get_session_history,

    input_messages_key="question",
```

```
    history_messages_key="history"
)



# ========================================
# 12. LOAD REGISTERED USERS
# ========================================

users = load_users()

current_user = None



# ========================================
# 13. START APPLICATION
# ========================================

print("\nMULTI-USER AI CHATBOT")
```

```
while True:

    # USER NOT LOGGED IN

    if current_user is None:

        print("\n1. Signup")
        print("2. Login")
        print("3. Exit")

        choice = input(
            "Enter Choice: "
        )

        if choice == "1":

            signup(users)

        elif choice == "2":
```

```
        current_user = login(
            users
        )

    elif choice == "3":

        break

    else:

        print("Invalid Choice")


# USER LOGGED IN

else:

    question = input(
        f"\n{current_user}: "
    ).strip()
```

```
if question.lower() == "logout":

    print("Logged Out")

    current_user = None



elif question.lower() == "exit":

    break



elif question:

    name = users[
        current_user
    ]["name"]
```

```
try:

    response = chatbot.invoke(
        {
            "name": name,
            "question": question
        },

        config={
            "configurable": {
                "session_id":
                current_user
            }
        }
    )

    print(
        "\nAI:",
        response.content
    )
```

```
        except Exception as error:

            print(
                "Error:",
                error
            )


print("\nProgram Ended")
```

---

4. Main Storage Design

We use two storage locations.

User Information

USERS_FILE = "users.json"

This file stores registered-user information.

Conceptually:

```
{
  "durga": {
    "name": "Durga",
```

```
    "password": "..."
  },

  "ravi": {
    "name": "Ravi",
    "password": "..."
  }
}
```

---

5. Chat History Folder

HISTORY_FOLDER = "chat_history"

The folder may contain:

```
chat_history/
    │
    ├── durga.json
    ├── ravi.json
    └── suresh.json
```

So:

Username

    ↓

Session ID

↓

History File

---

6. Create History Folder

```
os.makedirs(
    HISTORY_FOLDER,
    exist_ok=True
)
```

Meaning:

chat_history folder missing

↓

Create it

If already available:

Folder Exists

↓

Continue

---

7. load_users()

```
def load_users():
```

```
    if not os.path.exists(USERS_FILE):
        return {}

    with open(USERS_FILE, "r") as file:
        return json.load(file)
```

Purpose:

users.json

↓

Read Registered Users

↓

Return Python Dictionary

---

8. What If users.json Does Not Exist?

```
if not os.path.exists(
    USERS_FILE
):
    return {}
```

Meaning:

No users.json

↓

No Registered Users Yet

↓

Return Empty Dictionary

---

9. save_users()

```
def save_users(users):

    with open(
        USERS_FILE,
        "w"
    ) as file:

        json.dump(
            users,
            file,
            indent=2
        )
```

Purpose:

Python Users Dictionary

↓

json.dump()

↓

users.json

---

10. Signup Flow

Signup

↓

Enter Name

↓

Enter Username

↓

Enter Password

↓

Validate Input

↓

Check Existing Username

↓

Add User

↓

Save users.json

---

11. signup(users)

def signup(users):

This function receives the current users dictionary.

It asks for:

Name

Username

Password

---

12. .strip()

Example:

```
name = input(
    "Enter Name: "
).strip()
```

strip() removes unnecessary spaces from the beginning and end.

Example:

" Durga "

↓

"Durga"

---

13. .lower()

username = input(

"Enter Username: "

).strip().lower()

Example:

DURGA

Durga

durga

all become:

durga

This gives a consistent username format.

---

14. Validate Empty Fields

if not name or not username or not password:

print(

"All fields are required"

)

return

If any field is empty:

Signup Stops

---

15. Username Validation

```
if not username.replace(
    "_",
    ""
).isalnum():
```

The program allows:

Letters

Numbers

Underscores

Example:

durga123

durga_123

student1

---

16. Why replace("_", "")?

Suppose:

durga_123

isalnum() alone does not treat _ as alphanumeric.

So:

```
username.replace(
    "_",
    ""
)
```

converts:

durga_123

↓

durga123

Then:

.isalnum()

can validate letters and numbers.

---

17. Check Existing Username

if username in users:

```
    print(
        "Username already exists"
    )

    return
```

This prevents duplicate usernames.

---

18. Store New User

```
users[username] = {
    "name": name,
    "password": password
}
```

Example:

```
users["durga"] = {
    "name": "Durga",
    "password": "..."
}
```

Then:

```
save_users(users)
```

writes updated data to:

users.json

---

19. Login Flow

Login

↓

Enter Username

↓

Enter Password

↓

Username Exists?

↓

Password Matches?

↓

Login Successful

↓

Return Username

---

20. login(users)

def login(users):

The function returns either:

Username

or:

None

---

21. Invalid Username

if username not in users:

```
    print(
        "Invalid Username"
    )

    return None
```

So:

Unknown Username

↓

Login Failed

---

22. Password Check

if users[username]["password"] == password:

If matching:

Login Successful

Then:

return username

---

23. Why Return Username?

The returned username becomes:

current_user

Example:

current_user = "durga"

This tells the application:

Who Is Currently Logged In?

---

24. FileChatMessageHistory

We define:

class FileChatMessageHistory(

    BaseChatMessageHistory

):

This class handles persistent chat history.

Its main responsibilities are:

Read History

Save History

Clear History

---

25. Session ID Becomes File Name

```
self.file_path = os.path.join(
    HISTORY_FOLDER,
    session_id + ".json"
)
```

Suppose:

```
session_id = "durga"
```

Then:

```
chat_history/durga.json
```

Suppose:

```
session_id = "ravi"
```

Then:

```
chat_history/ravi.json
```

---

26. Very Important Design Decision

In this application:

USERNAME

=

SESSION ID

Example:

Username

durga

↓

session_id

durga

↓

History File

durga.json

So one registered user gets one conversation-history file.

---

27. messages Property

@property

def messages(self):

This loads existing conversation history.

If file does not exist:

return []

Meaning:

New User Conversation

↓

Empty History

---

28. Read Existing History

```
with open(
    self.file_path,
    "r"
) as file:


    data = json.load(file)
```

Flow:

JSON File

↓

json.load()

↓

Python Data

---

29. Restore LangChain Messages

return messages_from_dict(

data

)

Flow:

Python Dictionaries

↓

messages_from_dict()

↓

HumanMessage / AIMessage

---

30. add_messages()

def add_messages(

self,

messages

):

This method saves new conversation messages.

---

31. Combine Old and New Messages

```
all_messages = (
    self.messages
    + messages
)
```

Concept:

```
Old History
    +
New Messages
    ↓
Complete Updated History
```

---

32. Serialize Messages

```
data = [
    message_to_dict(message)
    for message in all_messages
]
```

Meaning:

```
LangChain Messages
        ↓
message_to_dict()
```

↓

Python Dictionaries

---

33. Save History

```
with open(
    self.file_path,
    "w"
) as file:


    json.dump(
        data,
        file,
        indent=2
    )
```

Flow:

Python Dictionaries

↓

json.dump()

↓

JSON File

---

34. Clear History

```
def clear(self):


    with open(
        self.file_path,
        "w"
    ) as file:


        json.dump(
            [],
            file
        )
```

This changes the stored history to:

```
[]
```

---

35. Create Model

```
llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
```

)

The same model can serve multiple users.

Important:

Same Model

≠

Same Conversation

The conversation is separated using:

session_id

---

36. Create Prompt

```
prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a helpful AI assistant. "
            "Answer in simple English. "
            "The user's name is {name}."
        ),

        MessagesPlaceholder(
```

```
            "history"
        ),

        (
            "human",
            "{question}"
        )
    ]
)
```

Prompt architecture:

System Message

↓

User Name

↓

Previous History

↓

Current Question

↓

Model

---

37. Dynamic User Name

The system prompt contains:

The user's name is {name}.

At runtime:

```
{
   "name": name,
   "question": question
}
```

supplies the actual value.

Example:

{name}

↓

Durga

So the final instruction can contain:

The user's name is Durga.

---

38. MessagesPlaceholder

```
MessagesPlaceholder(
   "history"
)
```

This is where previous conversation history is inserted.

Concept:

System Instruction

↓

Previous Human/AI Messages

↓

Current Question

---

39. LCEL Chain

```
chain = (
   prompt
   | llm
)
```

This means:

Prompt

↓

Model

---

40. get_session_history()

```
def get_session_history(
```

```
    session_id
):

    return FileChatMessageHistory(
        session_id
    )
```

Suppose:

session_id = "durga"

Then:

FileChatMessageHistory("durga")

uses:

chat_history/durga.json

---

41. RunnableWithMessageHistory

```
chatbot = RunnableWithMessageHistory(

    chain,

    get_session_history,
```

```
    input_messages_key="question",

    history_messages_key="history"
)
```

This adds conversation-history management around the LCEL chain.

---

42. input_messages_key

input_messages_key="question"

means:

Which Input Contains

Current User Message?

↓

question

Invocation:

```
{
    "name": name,
    "question": question
}
```

---

43. history_messages_key

history_messages_key="history"

matches:

MessagesPlaceholder(

"history"

)

So:

Previous History

↓

"history"

↓

MessagesPlaceholder

---

44. Load Registered Users

users = load_users()

This loads the contents of:

users.json

into the Python dictionary:

users

---

45. current_user

current_user = None

Initially:

No User Logged In

After successful login:

current_user = "durga"

---

46. Main Application Loop

while True:

The application continuously works until the user chooses to exit.

There are two major states:

1. User Not Logged In

2. User Logged In

---

47. State 1 — User Not Logged In

if current_user is None:

Then show:

1. Signup

2. Login

3. Exit

---

48. Signup Choice

if choice == "1":

    signup(users)

Flow:

Choose Signup

↓

signup(users)

↓

Register User

↓

Save users.json

---

49. Login Choice

elif choice == "2":

    current_user = login(

users

)

If login succeeds:

current_user

↓

Username

If login fails:

current_user

↓

None

---

50. Exit Choice

elif choice == "3":

break

This exits the main application loop.

---

51. State 2 — User Logged In

Once:

current_user != None

the program enters chatbot mode.

Example:

durga:

The username is displayed as the prompt.

---

52. Chat Input

```
question = input(
    f"\n{current_user}: "
).strip()
```

Example:

durga: What is Python?

---

53. Logout

If user enters:

logout

then:

current_user = None

So the program returns to:

Signup / Login / Exit Menu

---

54. Logout Flow

Logged-In User

↓

Types "logout"

↓

current_user = None

↓

Return to Main Menu

---

55. Exit

If user enters:

exit

the application ends.

---

56. Get User's Name

name = users[

current_user

]["name"]

Suppose:

current_user = "durga"

and:

```
users["durga"] = {
   "name": "Durga",
   "password": "..."
}
```

Then:

name

↓

Durga

---

57. Chatbot Invocation

```
response = chatbot.invoke(
   {
      "name": name,
      "question": question
   },

   config={
      "configurable": {
         "session_id":
```

```
        current_user
      }
    }
)
```

This contains two different types of information.

Prompt Data

```
{
   "name": name,
   "question": question
}
```

Session Configuration

```
config={
   "configurable": {
      "session_id": current_user
   }
}
```

---

58. Very Important Distinction

name

↓

Used Inside Prompt

question

↓

Current User Message

session_id

↓

Selects Conversation History

---

59. Username as Session ID

This program uses:

"session_id": current_user

Therefore:

current_user

↓

Username

↓

Session ID

Example:

durga

↓

Session ID = durga

↓

chat_history/durga.json

---

60. Multi-User Flow

Suppose Durga logs in:

Username: durga

↓

Session ID: durga

↓

durga.json

Ravi logs in:

Username: ravi

↓

Session ID: ravi

↓

ravi.json

So:

Durga Chat

↓

durga.json

and:

Ravi Chat

↓

ravi.json

do not use the same history file.

---

61. Complete Durga Flow

Durga Signup

↓

users.json

↓

Durga Login

↓

current_user = "durga"

↓

Ask Question

↓

session_id = "durga"

↓

Load durga.json

↓

Previous Durga History

+

Current Question

↓

Model

↓

AI Response

↓

Update durga.json

---

62. Complete Ravi Flow

Ravi Login

↓

current_user = "ravi"

↓

Ask Question

↓

session_id = "ravi"

↓

Load ravi.json

↓

Previous Ravi History

+

Current Question

↓

Model

↓

AI Response

↓

Update ravi.json

---

63. Application Storage Example

After registering two users:

users.json

conceptually contains:

```
{
  "durga": {
    "name": "Durga",
```

```
  "password": "..."
 },

 "ravi": {
  "name": "Ravi",
  "password": "..."
 }
}
```

And the folder may contain:

```
chat_history/
   |
   ├── durga.json
   └── ravi.json
```

---

64. Two Types of Persistence

This program demonstrates two different persistent data types.

User Persistence

User Details

```
   ↓
```

users.json

Conversation Persistence

Chat Messages

↓

chat_history/<username>.json

---

65. Why Use Separate Files?

If all users share one history file:

Durga Messages

+

Ravi Messages

+

Suresh Messages

may get mixed.

Instead:

durga.json

ravi.json

suresh.json

keeps them separated.

---

66. Why No store = {} Here?

Earlier, with:

InMemoryChatMessageHistory

we used:

store = {}

But here:

Session ID

↓

JSON File

is used for history.

So the file system becomes our history storage.

---

67. In-Memory vs This Application

In-Memory

session_id

↓

Python Dictionary

↓

InMemoryChatMessageHistory

This Application

session_id

↓

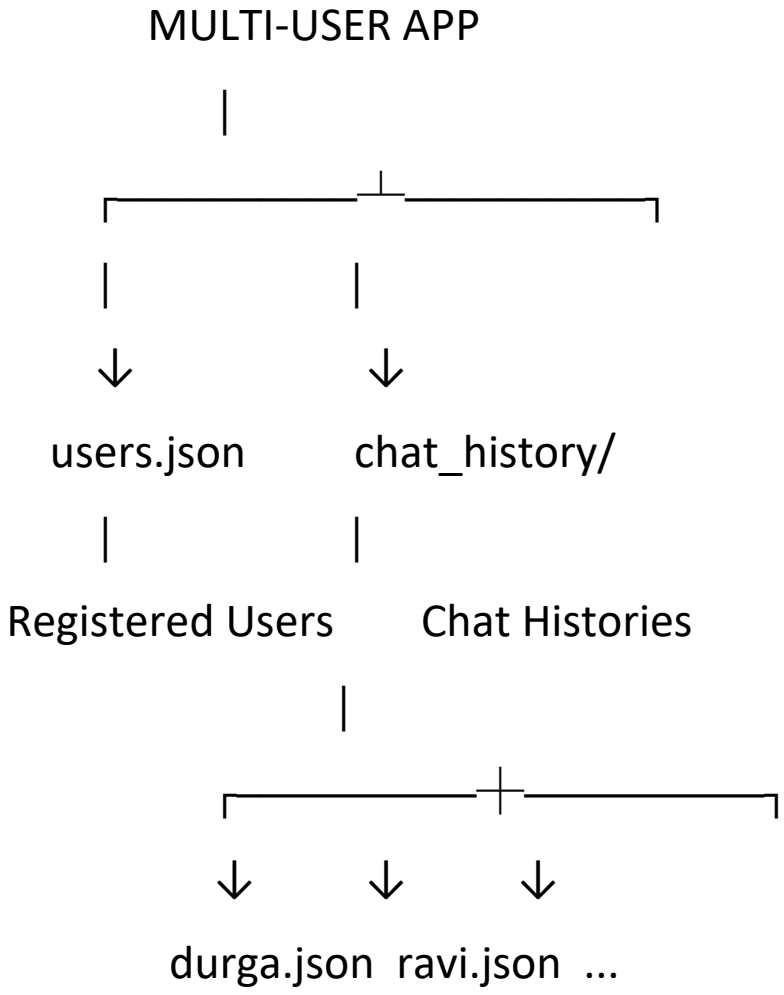
FileChatMessageHistory

↓

JSON File

---

68. Persistent Multi-User Architecture



---

69. Error Handling

The chatbot invocation is inside:

try:

and:

except Exception as error:

If something goes wrong:

print(

"Error:",

error

)

This prevents the application from immediately crashing because of an unhandled chatbot-call error.

---

70. Main Application States

Think of the application like this:

START

↓

Load Users

↓

current_user = None

↓

MAIN LOOP

↓

Logged In?

/ \

No Yes

↓ ↓

Signup Chat

Login Logout

Exit Exit

---

71. Full Program Flow

START

↓

Create chat_history folder

↓

Load users.json

↓

Show Signup/Login Menu

↓

User Logs In

↓

current_user = username

↓

Read User Question

↓

Get User Name

↓

Use Username as session_id

↓

Load User's JSON Chat History

↓

Insert History into Prompt

↓

Send Prompt to Model

↓

Get AI Response

↓

Save Updated Chat History

↓

Continue Chat

↓

Logout / Exit

---

72. Important Key Points

1. users.json stores registered-user information.
2. chat_history stores separate conversation-history files.
3. load_users() reads registered users.
4. save_users() writes updated users.
5. signup() creates a new user.
6. login() checks the provided login information.
7. current_user tells us who is logged in.
8. FileChatMessageHistory provides persistent message history.
9. Every username acts as a session ID in this program.
10. Every username gets a separate history JSON file.
11. MessagesPlaceholder("history") inserts previous conversation messages.
12. input_messages_key="question" tells the wrapper which field contains the current question.
13. history_messages_key="history" identifies where previous history belongs.

14. RunnableWithMessageHistory manages conversation history around the LCEL chain.
15. User details and conversation history are stored separately.

---

73. Five Most Important Connections

Connection 1

users.json

↓

Registered Users

Connection 2

current_user

↓

Logged-In Username

Connection 3

current_user

↓

session_id

Connection 4

session_id

↓

<username>.json

Connection 5

History File

↓

RunnableWithMessageHistory

↓

Context-Aware Chatbot

---

74. MCQ 1

What does users.json store in this program?

A. Only AI responses
B. Registered-user information
C. Model weights
D. Embeddings

Answer: B

Conclusion

users.json is used for registered-user data.

---

75. MCQ 2

What acts as the session_id?

A. Model name
B. Username of the logged-in user
C. User's question
D. Password

Answer: B

Conclusion

The current username identifies the user's conversation history.

---

76. MCQ 3

If the logged-in username is:

durga

which history file is used?

A. ravi.json
B. users.json
C. durga.json
D. history.txt

Answer: C

Conclusion

The username is used to construct the session-specific chat-history filename.

---

77. MCQ 4

What is the purpose of:

```
MessagesPlaceholder(
    "history"
)
```

A. Store passwords
B. Insert previous conversation messages into the prompt
C. Create users
D. Create API key

Answer: B

---

78. MCQ 5

What happens when the user types:

```
logout
```

A. All users are deleted
B. Chat history is cleared
C. current_user becomes None
D. Program always ends

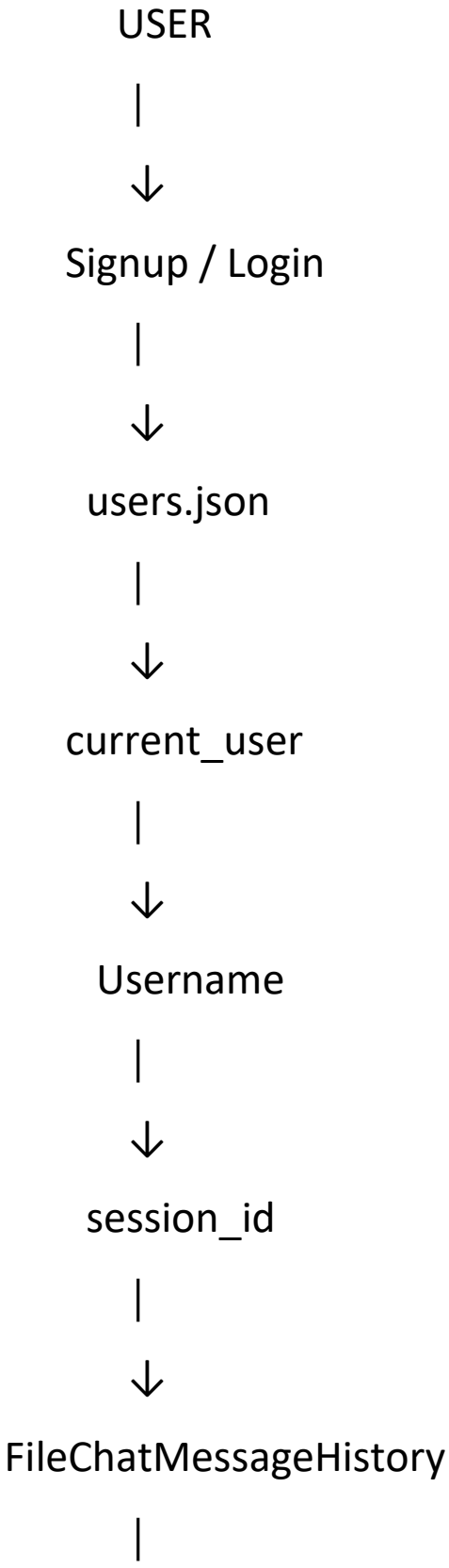
Answer: C

Conclusion

The application returns to the Signup/Login menu.

---

79. Final Master Architecture


USER
Signup / Login
users.json
current_user
Username
session_id
FileChatMessageHistory

↓

chat_history/username.json

|

↓

Previous Messages

|

↓

Prompt + Question

|

↓

LLM

|

↓

AI Response

|

↓

Save Updated History

---

80. Easy Memory Formula

USER ACCOUNT

↓

users.json

USER CONVERSATION

↓

username.json

And:

Username

↓

Session ID

↓

Separate Persistent History

---

81. One-Line Conclusion

This multi-user chatbot stores registered users in users.json, uses the logged-in username as the session_id, and stores each user's conversation in a separate JSON history file so different users can maintain separate persistent conversations.